NEXT ENGINEER · _SHEERAT_

Artificial Intelligence

BCS701

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT | **Semester:** 7 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 4th year, Core, 2025-26 onwards

UNIT 1 Introduction to AI and Intelligent Agents

UNIT 2 Problem Solving and Search Strategies

UNIT 3 Knowledge Representation and Reasoning

UNIT 4 Uncertainty and Learning Techniques

UNIT 5 Applications of AI and Multi-Agent Systems

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: agents, algorithms and comparison tables.
3. **Solved examples**: the numericals and traces that appear every year (BFS/DFS, A* and greedy search, minimax with alpha-beta, CSP, resolution, unification, Bayes rule and Bayesian networks, fuzzy sets, perceptron learning).
4. **Code / key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (AI definitions, agents, PEAS, environment types, agent architectures, problem formulation) and Unit 2 (BFS, DFS, IDS, greedy, A*, hill climbing, simulated annealing, CSP, minimax, alpha-beta).
- **Afternoon (3 hours):** Unit 3 (propositional and first-order logic, Wumpus world, Prolog, forward and backward chaining, resolution, ontologies) and Unit 4 (probability, Bayes rule, Bayesian networks, fuzzy logic, perceptron, backpropagation, supervised and unsupervised learning).
- **Evening (2 hours):** Unit 5 (NLP, information retrieval and extraction, robotics, speech recognition, software agents, multi-agent negotiation, explainable AI). Then the "Last-hour checklist".
- **In the exam:** for search questions draw the tree or graph and write the frontier at each step; for logic questions convert to clause form first; for Bayesian networks write the chain-rule product before substituting numbers.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to AI and Intelligent Agents

✍️ 2-mark definitions

- **Artificial intelligence:** the study and design of computer systems that perform tasks which normally need human intelligence, such as reasoning, learning, perception, language understanding and decision making.
- **Four approaches to AI (Russell and Norvig):** thinking humanly (cognitive modelling), acting humanly (Turing test), thinking rationally (laws of thought, logic), acting rationally (rational agent approach, the one AIMA follows).
- **Turing test (1950):** a machine passes if a human interrogator, communicating by text, cannot reliably tell it from a human. It needs natural language processing, knowledge representation, automated reasoning and machine learning (the total Turing test also needs vision and robotics).
- **Agent:** anything that perceives its environment through sensors and acts on it through actuators.
- **Percept sequence:** the complete history of everything the agent has perceived.
- **Agent function:** a mapping from percept sequences to actions; the **agent program** implements it on a physical architecture (agent = architecture + program).
- **Rational agent:** an agent that, for each percept sequence, selects the action expected to maximize its performance measure, given its percepts and built-in knowledge.
- **PEAS:** the task environment description: Performance measure, Environment, Actuators, Sensors.
- **Autonomy:** an agent's ability to rely on its own percepts and learning rather than only on built-in knowledge.
- **State space:** the set of all states reachable from the initial state by any sequence of actions.
- **Problem formulation:** defining the initial state, actions, transition model, goal test and path cost.

💡 Core concepts

History of AI (landmarks)

Year	Event
1943	McCulloch and Pitts model of artificial neurons
1950	Turing's paper "Computing Machinery and Intelligence" (Turing test)
1956	Dartmouth workshop: John McCarthy coins "artificial intelligence"
1958–1970	LISP (McCarthy), General Problem Solver, perceptron (Rosenblatt), early optimism
1970s	First "AI winter"; expert systems begin (DENDRAL, MYCIN)
1980s	Expert systems boom (R1/XCON), backpropagation popularized (1986); second winter in the late 1980s
1997	IBM Deep Blue defeats Garry Kasparov at chess

Year	Event
2011	IBM Watson wins Jeopardy!; Siri launched
2012	Deep learning breakthrough in image recognition (AlexNet on ImageNet)
2016	DeepMind's AlphaGo defeats Lee Sedol at Go
2017 onwards	Transformer architecture; large language models and generative AI (ChatGPT, 2022)

Scope and applications: game playing, natural language processing (chatbots, translation), computer vision (face recognition, medical imaging), robotics (industrial robots, self-driving cars), expert systems (diagnosis), speech recognition (voice assistants), recommendation systems, fraud detection, agriculture (crop monitoring), education (adaptive tutoring), healthcare (drug discovery).

Characteristics of intelligent agents: autonomy, reactivity (respond to changes), pro-activeness (goal-directed behaviour), social ability (communicate and cooperate), learning (improve with experience), rationality.

Types of environments

Property	Options	Example
Observability	Fully vs partially observable	Chess (fully); poker, taxi driving (partially)
Agents	Single vs multi-agent	Crossword (single); chess, driving (multi)
Determinism	Deterministic vs stochastic	Chess (deterministic); dice games, driving (stochastic)
Episodes	Episodic vs sequential	Image classification (episodic); chess (sequential)
Change	Static vs dynamic (semi-dynamic)	Crossword (static); taxi driving (dynamic); chess with a clock (semi-dynamic)
States and time	Discrete vs continuous	Chess (discrete); driving (continuous)
Knowledge	Known vs unknown (rules of the environment)	Known game rules vs a new video game

Types of agents

1. **Simple reflex agent:** acts only on the current percept using condition-action rules (if car in front is braking then brake). Works only in fully observable environments; can get stuck in loops.
2. **Model-based reflex agent:** keeps an internal state (a model of how the world evolves and how actions affect it) to handle partial observability.
3. **Goal-based agent:** chooses actions that achieve a goal, using search and planning; more flexible.
4. **Utility-based agent:** chooses actions that maximize expected utility (a measure of how desirable a state is); handles trade-offs and uncertainty.
5. **Learning agent:** has a learning element (improves), performance element (selects actions), critic (gives feedback against a standard), and problem generator (suggests exploratory actions).

Agent architectures: reactive (subsumption architecture, Brooks), deliberative (symbolic reasoning, BDI: beliefs, desires, intentions), hybrid (layered: reactive layer for fast responses, deliberative layer for

planning), and learning architectures.

Problem-solving approach

1. Goal formulation.
2. Problem formulation: initial state, actions, transition model, goal test, path cost.
3. Search for a solution (sequence of actions).
4. Execution. - A **problem-solving agent** is a goal-based agent that uses atomic state representations and searches the state space.

Example problems

- **Toy problems:** vacuum world (2 squares → 8 states), 8-puzzle (9!/2 = 181,440 reachable states), 8-queens, water jug, missionaries and cannibals, Tower of Hanoi.
- **Real-world problems:** route finding (maps, GPS), travelling salesperson, VLSI layout, robot navigation, automatic assembly sequencing, protein design, internet search.

🧮 Solved examples

Example 1 (PEAS for an automated taxi).

- Performance: safe, fast, legal, comfortable trip, maximize profit.
- Environment: roads, traffic, pedestrians, customers, weather.
- Actuators: steering, accelerator, brake, signal, horn, display.
- Sensors: cameras, LIDAR, radar, GPS, speedometer, odometer, engine sensors, microphone.
- Environment type: partially observable, multi-agent, stochastic, sequential, dynamic, continuous.

Example 2 (water jug formulation). Jugs of 4 L and 3 L, no markings; get exactly 2 L in the 4 L jug.

- State (x, y): litres in the 4 L and 3 L jugs; initial (0, 0); goal (2, n) for any n.
- Actions: fill either jug, empty either jug, pour one into the other until the source is empty or the target is full.
- One solution: (0, 0) → (0, 3) → (3, 0) → (3, 3) → (4, 2) → (0, 2) → (2, 0). Six steps.

Example 3 (8-puzzle formulation). States: positions of 8 tiles and the blank; initial: any configuration; actions: move the blank Left, Right, Up, Down; goal test: matches the goal configuration; path cost: 1 per move. Branching factor about 3.

Key points to remember

- Rational agent maximizes expected performance; PEAS describes the task.
- Environment properties: observable, agents, deterministic, episodic, static, discrete, known.
- Five agent types: simple reflex, model-based, goal-based, utility-based, learning.
- Problem = initial state, actions, transition model, goal test, path cost.

⭐ Important questions

1. Define AI. Explain its scope, history and applications. What is the Turing test?
2. What is an intelligent agent? Explain the characteristics of agents and the concept of rationality.

NEXT ENGINEER _sheerat_

3. Explain the types of environments with examples. Write PEAS for a given agent.
4. Explain the types of agents with diagrams, including the learning agent.
5. Explain the problem-solving approach. Formulate the water jug, 8-puzzle or 8-queens problem.

NEXT ENGINEER
sheerat

UNIT 2 Problem Solving and Search Strategies

✍️ 2-mark definitions

- **Search strategy:** the order in which nodes of the search tree are expanded.
- **Completeness, optimality, time and space complexity:** criteria for judging search algorithms: does it always find a solution if one exists, does it find the lowest-cost solution, how long does it take, how much memory does it need.
- **Branching factor (b), depth of shallowest solution (d), maximum depth (m):** parameters used to express complexity.
- **Uninformed (blind) search:** uses only the problem definition, no knowledge about how close a state is to the goal.
- **Informed (heuristic) search:** uses a heuristic function h(n) estimating the cost from n to the goal.
- **Heuristic function h(n):** an estimate of the cheapest path cost from node n to a goal.
- **Admissible heuristic:** never overestimates the true cost to reach the goal.
- **Consistent (monotonic) heuristic:** $h(n) \le c(n, a, n') + h(n')$ for every successor n'; consistency implies admissibility.
- **Local search:** keeps only the current state and moves to neighbours; uses little memory; good for optimization.
- **Local maximum, plateau, ridge:** problems for hill climbing where no neighbour is better although the global optimum is elsewhere.
- **Constraint satisfaction problem (CSP):** variables, domains, and constraints; a solution assigns values to all variables satisfying all constraints.
- **Adversarial search:** search in competitive multi-agent environments (games).
- **Minimax:** chooses the move with the best value assuming the opponent also plays optimally.
- **Alpha-beta pruning:** removes branches that cannot influence the minimax decision; α = best value for MAX so far, β = best value for MIN so far.

💡 Core concepts

Uninformed search strategies

Algorithm	How it expands	Complete?	Optimal?	Time	Space
Breadth-first (BFS)	Shallowest node first (FIFO queue)	Yes (finite b)	Yes, if all step costs are equal	O(b^d)	O(b^d)
Uniform-cost (UCS)	Lowest path cost g(n) first (priority queue)	Yes (positive costs)	Yes	O(b^(1+C*/ε))	same
Depth-first (DFS)	Deepest node first (LIFO stack)	No (infinite paths, loops)	No	O(b^m)	O(bm)
Depth-limited	DFS with depth limit l	No if l < d	No	O(b^l)	O(bl)

Algorithm	How it expands	Complete?	Optimal?	Time	Space
Iterative deepening (IDS)	DFS with limits 0, 1, 2, ...	Yes	Yes (equal costs)	O(b^d)	O(bd)
Bidirectional	Search from start and goal, meet in the middle	Yes (with BFS)	Yes (equal costs)	O(b^(d/2))	O(b^(d/2))

IDS combines the low memory of DFS with the completeness of BFS; re-generating shallow nodes costs little because most nodes are at the deepest level.

Informed search

- **Greedy best-first search:** expands the node with the lowest h(n). Fast but not optimal and not complete in general (can loop).
- **A* search:** expands the node with the lowest f(n) = g(n) + h(n). Complete and optimal if h is admissible (tree search) or consistent (graph search). Time and space can still be exponential; memory is the main problem (solutions: IDA*, RBFS, SMA*).
- **Heuristics for the 8-puzzle:** h1 = number of misplaced tiles; h2 = sum of Manhattan distances. Both are admissible; h2 dominates h1 (h2 ≥ h1), so it expands fewer nodes.

Local search and optimization

- **Hill climbing (steepest ascent):** move to the best neighbour; stop when no neighbour is better. Problems: local maxima, plateaus, ridges. Variants: stochastic hill climbing, first-choice, random-restart hill climbing.
- **Simulated annealing:** pick a random neighbour; if better, move; if worse by ΔE, move with probability e^(ΔE/T), where temperature T decreases over time. Early on it escapes local maxima; at low T it behaves like hill climbing.
- **Local beam search** (keep k states) and **genetic algorithms** (population, selection, crossover, mutation).

Constraint satisfaction problems

- Components: variables X, domains D, constraints C (unary, binary, global such as AllDiff).
- Examples: map colouring, N-queens, Sudoku, scheduling, cryptarithmetic (SEND + MORE = MONEY).
- **Backtracking search:** assign one variable at a time, backtrack on failure.
- **Heuristics:** minimum remaining values (MRV, choose the variable with fewest legal values), degree heuristic (most constraints on remaining variables), least constraining value.
- **Inference:** forward checking (remove inconsistent values from neighbours' domains), arc consistency (AC-3), constraint propagation.

Game playing

- A game: initial state, player, actions, result, terminal test, utility function.
- **Minimax:** MAX takes the maximum of children's values, MIN the minimum; time O(b^m), space O(bm).
- **Alpha-beta pruning:** prune when α ≥ β. Same result as minimax; with perfect move ordering, time O(b^(m/2)), effectively doubling the searchable depth.
- **Imperfect real-time decisions:** cut off search at a depth and use an evaluation function (weighted sum of features such as material in chess).

NEXT ENGINEER _sheerat_

- **Stochastic games** (backgammon): add chance nodes; use **expectiminimax** (value of a chance node = probability-weighted average of children).
- **Partially observable games** (card games like bridge, poker; Kriegspiel): reason over belief states (sets of possible states); average over possible deals.

🧮 Solved examples

Example 1 (BFS and DFS order). Tree: A has children B, C; B has D, E; C has F, G. Goal G.

- BFS: **A, B, C, D, E, F, G** (7 nodes expanded/tested).
- DFS (left first): **A, B, D, E, C, F, G**.

Example 2 (A* vs greedy). Edges: S–A 1, S–B 4, A–B 2, A–C 5, B–C 2, B–G 6, C–G 3. Heuristic (consistent): h(S) = 7, h(A) = 6, h(B) = 4, h(C) = 2, h(G) = 0.

Step	Expand	g	f = g + h	Frontier after expansion (f)
1	S	0	7	A (7), B (8)
2	A	1	7	B via A (3 + 4 = 7), C via A (6 + 2 = 8), B (8)
3	B	3	7	C via B (5 + 2 = 7), G via B (9 + 0 = 9), …
4	C	5	7	G via C (8 + 0 = 8), G (9)
5	G	8	8	goal reached

A* path **S → A → B → C → G**, cost **8** (optimal). Greedy best-first follows the lowest h: S → B (h = 4) → G (h = 0), path cost **10**: found quickly but **not optimal**.

Example 3 (minimax with alpha-beta). MAX root with three MIN children whose leaves are (3, 12, 8), (2, 4, 6), (14, 5, 2).

- Minimax values: MIN nodes = 3, 2, 2; root = **3**.
- Alpha-beta: after the first MIN node, α = 3. In the second MIN node, the first leaf 2 gives β = 2 ≤ α = 3, so leaves **4 and 6 are pruned**. The third MIN node must examine 14, 5 and 2 (value 2). Root = 3; the best move is the first branch.

Example 4 (8-puzzle heuristics). Start: 7 2 4 / 5 _ 6 / 8 3 1; goal: _ 1 2 / 3 4 5 / 6 7 8. h1 (misplaced tiles) = **8**; h2 (Manhattan) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = **18** (tiles 1 to 8).

Example 5 (CSP: map colouring of Australia). Variables WA, NT, Q, NSW, V, SA, T; domain {red, green, blue}; adjacent regions differ. MRV + degree picks SA first (5 neighbours). One solution: SA = blue, WA = red, NT = green, Q = red, NSW = green, V = red, T = any.

Example 6 (simulated annealing). A worse move with ΔE = −2 is accepted at T = 10 with probability e^(−0.2) ≈ **0.82**, at T = 1 with e^(−2) ≈ **0.135**: as the temperature drops, bad moves become rare.

Key points to remember

- BFS: complete, optimal (equal costs), O(b^d) space. DFS: O(bm) space, not complete. IDS: best uninformed for large spaces.
- A*: f = g + h; optimal with admissible (tree) or consistent (graph) h.

- Hill climbing problems: local maxima, plateaus, ridges. Simulated annealing accepts worse moves with e^(ΔE/T).
- CSP: backtracking + MRV, degree, LCV, forward checking, AC-3.
- Minimax O(b^m); alpha-beta best case O(b^(m/2)); expectiminimax for chance.

⭐ Important questions

1. Compare BFS, DFS, uniform-cost, depth-limited, iterative deepening and bidirectional search (completeness, optimality, time, space).
2. Explain greedy best-first and A* search. Prove that A* is optimal with an admissible heuristic. Solve a numerical.
3. Explain hill climbing, its problems, and simulated annealing.
4. What is a CSP? Explain backtracking with heuristics and constraint propagation with an example.
5. Explain the minimax algorithm and alpha-beta pruning with an example. How are stochastic and partially observable games handled?

NEXT ENGINEER _sheerat_

UNIT 3 Knowledge Representation and Reasoning

✍️ 2-mark definitions

- **Knowledge base (KB):** a set of sentences in a formal language representing what the agent knows.
- **Knowledge-based agent:** an agent that TELLs its KB what it perceives, ASKs the KB what to do, and TELLs it what action it took.
- **Syntax and semantics:** syntax defines well-formed sentences; semantics defines their truth in each model (possible world).
- **Entailment (KB ⊨ α):** α is true in every model in which KB is true.
- **Inference (KB ⊢ α):** deriving α from KB by an algorithm; it is **sound** if it derives only entailed sentences and **complete** if it derives all entailed sentences.
- **Propositional logic:** logic of atomic propositions joined by connectives ¬, ∧, ∨, ⇒, ⇔.
- **First-order logic (FOL):** logic with objects, relations, functions, variables and quantifiers (∀ for all, ∃ there exists).
- **Validity and satisfiability:** a sentence is valid (a tautology) if true in all models; satisfiable if true in some model.
- **Horn clause:** a disjunction of literals with at most one positive literal (a rule or a fact), used in Prolog and chaining.
- **Forward chaining:** data-driven inference from known facts to new conclusions until the goal appears.
- **Backward chaining:** goal-driven inference that works back from the goal to find supporting facts.
- **Resolution:** an inference rule that combines two clauses containing complementary literals; refutation proves KB ⊨ α by deriving the empty clause from KB ∧ ¬α.
- **Unification:** finding a substitution that makes two logical expressions identical.
- **Ontology:** a formal description of the concepts in a domain, their properties and relationships (categories, objects, events).

💡 Core concepts

Wumpus world (PEAS): a 4 × 4 cave with the agent at [1,1], a wumpus (smells: stench in adjacent squares), pits (breeze in adjacent squares) and gold (glitter). Performance: +1000 for gold, −1000 for death, −1 per action, −10 for the arrow. Actions: forward, turn left/right, grab, shoot, climb. Sensors: stench, breeze, glitter, bump, scream. The agent uses logic: no breeze in [1,1] ⇒ [1,2] and [2,1] are safe; breeze in [2,1] ⇒ pit in [2,2] or [3,1]; combining facts locates pits and the wumpus.

Propositional logic inference

- Laws: De Morgan, distributivity, contraposition, implication elimination (P ⇒ Q ≡ ¬P ∨ Q), biconditional elimination.
- Rules: Modus Ponens (from P ⇒ Q and P infer Q), And-elimination, resolution.

- **CNF conversion:** eliminate ⇔ and ⇒, move ¬ inwards, distribute ∨ over ∧.

First-order logic

- Terms: constants (John), variables (x), functions (Mother(x)). Atomic sentences: predicates (Brother(John, Richard)). Quantifiers: ∀x King(x) ⇒ Person(x); ∃x Crown(x) ∧ OnHead(x, John).
- Common mistakes: use ⇒ with ∀ and ∧ with ∃.
- Inference: universal instantiation, existential instantiation (Skolem constants), Generalized Modus Ponens with unification.

Converting FOL to clause form (CNF)

1. Eliminate ⇔ and ⇒.
2. Move ¬ inwards (¬∀x P ≡ ∃x ¬P).
3. Standardize variables apart.
4. Skolemize (replace ∃ variables with Skolem constants or functions).
5. Drop universal quantifiers.
6. Distribute ∨ over ∧ and split into clauses.

Forward vs backward chaining

Forward chaining	Backward chaining
Data-driven: from facts to conclusions	Goal-driven: from goal to facts
Fires all rules whose premises are satisfied	Uses only rules relevant to the goal
May derive many irrelevant facts	Efficient for answering specific queries
Used in production systems, monitoring	Used in Prolog, diagnosis, theorem proving
Breadth-first style	Depth-first style

Prolog (logic programming): facts (`parent(tom, bob).`), rules (`grandparent(X, Z) :- parent(X, Y), parent(Y, Z).`) and queries (`?- grandparent(tom, Who).`). Prolog uses backward chaining with depth-first search, unification and backtracking; closed-world assumption and negation as failure.

Ontological engineering: building ontologies: upper ontology (general concepts: things, events, time, space), categories and objects (inheritance: Basketball ∈ Balls, Balls ⊂ Spheres), physical composition (PartOf), measurements, events and time (event calculus), mental objects (beliefs). Reasoning systems for categories: semantic networks (nodes and labelled links, inheritance) and description logics (formal languages like OWL, used in the semantic web). Examples: WordNet, Cyc, SNOMED CT.

🧮 Solved examples

Example 1 (propositional resolution). KB: P ⇒ Q, Q ⇒ R, P. Prove R.

- Clauses: (1) ¬P ∨ Q, (2) ¬Q ∨ R, (3) P, (4) ¬R (negated goal).
- Resolve (2) and (4): ¬Q. Resolve with (1): ¬P. Resolve with (3): **empty clause**. Contradiction, so KB ⊨ R.

Example 2 (FOL to CNF and resolution: "crime" example in brief). Facts: "It is a crime for an American to sell weapons to hostile nations. Nono, an enemy of America, has missiles, all sold to it by Colonel West, an American." Prove Criminal(West).

- Rules: American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) ⇒ Criminal(x); Missile(x) ∧ Owns(Nono, x) ⇒ Sells(West, x, Nono); Missile(x) ⇒ Weapon(x); Enemy(x, America) ⇒ Hostile(x).
- Facts: Owns(Nono, M1), Missile(M1), American(West), Enemy(Nono, America).
- Forward chaining: Weapon(M1), Sells(West, M1, Nono), Hostile(Nono) → **Criminal(West)**.

Example 3 (unification).

- Knows(John, x) and Knows(John, Jane): θ = {x/Jane}.
- Knows(John, x) and Knows(y, Mother(y)): θ = {y/John, x/Mother(John)}.
- Knows(John, x) and Knows(x, Elizabeth): fails (x cannot be both John and Elizabeth); standardizing apart (rename to Knows(z, Elizabeth)) gives {x/Elizabeth, z/John}.

Example 4 (Prolog).

```
parent(tom, bob).
parent(bob, ann).
parent(bob, pat).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
sibling(X, Y)     :- parent(P, X), parent(P, Y), X \= Y.

% ?- grandparent(tom, W).   W = ann ; W = pat.
% ?- sibling(ann, S).       S = pat.
```

Example 5 (English to FOL).

- "All students like AI": ∀x Student(x) ⇒ Likes(x, AI).
- "Some student failed": ∃x Student(x) ∧ Failed(x).
- "Everyone loves someone": ∀x ∃y Loves(x, y).
- "Nobody likes taxes": ¬∃x Likes(x, Taxes) ≡ ∀x ¬Likes(x, Taxes).

Key points to remember

- KB agent: TELL, ASK. Entailment vs inference; soundness and completeness.
- Wumpus world percepts: stench, breeze, glitter, bump, scream.
- CNF steps; Skolemization; unification (most general unifier).
- Forward chaining = data-driven; backward chaining = goal-driven (Prolog).
- Resolution refutation: add ¬goal, derive empty clause.
- Ontology: categories, inheritance, semantic networks, description logic.

⭐ Important questions

1. Explain knowledge-based agents and the Wumpus world.
2. Explain propositional logic: syntax, semantics, inference rules and resolution.
3. Explain first-order logic. Convert English sentences to FOL and FOL to CNF.
4. Explain forward and backward chaining with an example and compare them.
5. Explain resolution in FOL and unification. Prove a goal by resolution.
6. Write a short Prolog program and explain how Prolog answers queries.

7. What is ontological engineering? Explain categories, objects and semantic networks.

NEXT ENGINEER
sheerat

UNIT 4 Uncertainty and Learning Techniques

✍️ 2-mark definitions

- **Uncertainty:** the agent cannot know for sure the state of the world or the result of its actions, because of partial observability, noisy sensors or complexity.
- **Prior (unconditional) probability:** P(A), the degree of belief without other evidence.
- **Conditional (posterior) probability:** P(A | B) = P(A ∧ B)/P(B).
- **Bayes' rule:** P(H | E) = P(E | H) P(H)/P(E).
- **Conditional independence:** X and Y are independent given Z if P(X, Y | Z) = P(X | Z) P(Y | Z).
- **Bayesian network:** a directed acyclic graph whose nodes are random variables, with a conditional probability table (CPT) for each node given its parents; it represents the full joint distribution compactly.
- **Fuzzy set:** a set where each element has a membership degree between 0 and 1.
- **Fuzzy logic:** reasoning with degrees of truth rather than true/false, to handle imprecise concepts such as "tall" or "hot".
- **Perceptron:** a single artificial neuron that computes a weighted sum of inputs plus bias and applies a step function; it can learn linearly separable functions.
- **Backpropagation:** the algorithm that trains multi-layer neural networks by propagating the error backwards and updating weights by gradient descent.
- **Machine learning:** improving performance at a task through experience (data) without explicit programming.
- **Supervised learning:** learning a mapping from inputs to known outputs (labels): classification and regression.
- **Unsupervised learning:** finding structure in unlabelled data: clustering, dimensionality reduction.

💡 Core concepts

Probabilistic reasoning: full joint distribution grows as 2^n for n Boolean variables; using independence and conditional independence reduces it. Naive Bayes assumes effects are independent given the cause: P(Cause, E1, ..., En) = P(Cause) Π P(Ei | Cause).

Bayesian networks

- Each node X has P(X | Parents(X)).
- **Chain rule:** P(x1, ..., xn) = Π P(xi | parents(Xi)).
- Size: a Boolean node with k parents needs 2^k numbers, instead of 2^n for the full joint.
- Inference: exact (enumeration, variable elimination) or approximate (sampling: rejection sampling, likelihood weighting, Gibbs sampling).
- **Burglary network (AIMA):** Burglary (B) and Earthquake (E) → Alarm (A) → JohnCalls (J), MaryCalls (M). P(B) = 0.001, P(E) = 0.002, P(A | B, E) = 0.95, P(A | B, ¬E) = 0.94, P(A | ¬B, E) = 0.29, P(A | ¬B, ¬E) = 0.001, P(J | A) = 0.90, P(J | ¬A) = 0.05, P(M | A) = 0.70, P(M | ¬A) = 0.01.

Fuzzy logic

- Operations: union $\mu(A \cup B) = \max(\mu A, \mu B)$; intersection $\mu(A \cap B) = \min(\mu A, \mu B)$; complement $\mu(\neg A) = 1 - \mu A$.
- Membership functions: triangular, trapezoidal, Gaussian.
- **Fuzzy inference system (Mamdani):** fuzzification of crisp inputs → rule evaluation (IF temperature is hot AND humidity is high THEN fan speed is fast) → aggregation → defuzzification (centroid method) to a crisp output.
- Applications: washing machines, air conditioners, camera autofocus, braking systems.

Neural networks (basics)

- **Perceptron:** $y = \text{step}(\Sigma\, w_i x_i + b)$. Learning rule: $w_i \leftarrow w_i + \eta\,(t - y)\,x_i$; $b \leftarrow b + \eta\,(t - y)$. Converges for linearly separable data (AND, OR) but not for XOR (Minsky and Papert, 1969).
- **Multi-layer perceptron:** input layer, hidden layers with non-linear activations (sigmoid, tanh, ReLU), output layer; can represent non-linear functions like XOR.
- **Backpropagation:** forward pass computes outputs; error $E = \tfrac{1}{2} \Sigma\,(t - y)^2$; backward pass computes the gradient of E for each weight using the chain rule; update $w \leftarrow w - \eta\, \partial E/\partial w$. For a sigmoid output unit, $\delta = (t - y)\,y\,(1 - y)$; for hidden units, $\delta_j = y_j\,(1 - y_j)\,\Sigma\, w_{jk}\,\delta_k$.

Machine learning in the AI context

- Learning agents improve their performance element from feedback.
- **Supervised:** decision trees, k-nearest neighbours, naive Bayes, linear and logistic regression, SVM, neural networks. Needs labelled training data; evaluated on a test set (accuracy, precision, recall); risk of overfitting.
- **Unsupervised:** k-means clustering, hierarchical clustering, PCA, association rules.
- **Reinforcement learning:** an agent learns from rewards and penalties by interacting with the environment (Q-learning).

🧮 Solved examples

Example 1 (Bayes' rule). A disease affects 1% of people. A test detects it 90% of the time (sensitivity) and gives a false positive 5% of the time. $P(\text{disease} \mid \text{positive}) = (0.9 \times 0.01)/(0.9 \times 0.01 + 0.05 \times 0.99) = 0.009/0.0585 \approx$ **0.154 (15.4%)**. Most positives are false because the disease is rare.

Example 2 (Bayesian network). Probability that both John and Mary call, the alarm sounds, and there is neither burglary nor earthquake: $P(j, m, a, \neg b, \neg e) = P(j \mid a)\,P(m \mid a)\,P(a \mid \neg b, \neg e)\,P(\neg b)\,P(\neg e) = 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998 \approx$ **0.000628**.

Example 3 (fuzzy sets). $A = \{0.2/x1, 0.7/x2, 1.0/x3\}$, $B = \{0.5/x1, 0.4/x2, 0.6/x3\}$.

- $A \cup B = \{0.5/x1, 0.7/x2, 1.0/x3\}$; $A \cap B = \{0.2/x1, 0.4/x2, 0.6/x3\}$; $\neg A = \{0.8/x1, 0.3/x2, 0/x3\}$.

Example 4 (perceptron learning AND). Initial $w1 = w2 = b = 0$, $\eta = 1$, output 1 if $w \cdot x + b > 0$.

- Epoch 1: (0,0) → 0 ✓; (0,1) → 0 ✓; (1,0) → 0 ✓; (1,1) → 0 but target 1 → $w = (1, 1)$, $b = 1$.
- Training continues; after 6 epochs it converges to **$w = (2, 1)$, $b = -2$**. Check: (1,1): $2 + 1 - 2 = 1 > 0$ → 1; (1,0): 0 → 0; (0,1): −1 → 0; (0,0): −2 → 0. All correct.

NEXT ENGINEER _sheerat_

Example 5 (why XOR needs a hidden layer). XOR outputs 1 for (0,1) and (1,0) and 0 for (0,0) and (1,1). No single straight line separates these points, so one perceptron cannot learn it. Two hidden units (one computing OR, one computing NAND) followed by an AND unit solve it.

Key points to remember

- Bayes: P(H | E) = P(E | H) P(H)/P(E); base-rate effect (15.4% example).
- Bayesian network chain rule; CPT size 2^k per Boolean node.
- Fuzzy: max, min, 1 − μ; Mamdani: fuzzify, rules, aggregate, defuzzify.
- Perceptron rule w ← w + η(t − y)x; fails on XOR.
- Backpropagation: forward pass, error, backward gradients, weight update.
- Supervised (labelled; classification, regression) vs unsupervised (clustering).

⭐ Important questions

1. Why is uncertainty a problem for agents? Explain probabilistic reasoning and Bayes' rule with an example.
2. What is a Bayesian network? Explain its construction and inference, and solve a numerical.
3. Explain fuzzy logic: fuzzy sets, operations, membership functions and the fuzzy inference process.
4. Explain the perceptron and its learning rule. Why can a perceptron not learn XOR?
5. Explain the backpropagation algorithm (introductory).
6. Explain supervised and unsupervised learning with examples.

UNIT 5 Applications of AI and Multi-Agent Systems

✍️ 2-mark definitions

- **Natural language processing (NLP):** enabling computers to understand, interpret and generate human language.
- **Language model:** a probability distribution over sequences of words; n-gram models estimate P(word | previous n − 1 words).
- **Machine translation (MT):** automatically translating text from one language to another (rule-based, statistical, neural).
- **Information retrieval (IR):** finding documents relevant to a user's query from a large collection (search engines).
- **Information extraction (IE):** extracting structured information (names, dates, relations) from unstructured text.
- **Precision and recall:** precision = relevant retrieved / all retrieved; recall = relevant retrieved / all relevant.
- **Robot:** a physical agent that perceives its environment through sensors and manipulates it with effectors.
- **Speech recognition:** converting spoken language into text.
- **Software agent:** a program that acts autonomously on behalf of a user or another program (for example a shopping bot or a scheduling assistant).
- **Multi-agent system (MAS):** a system of several interacting agents that may cooperate or compete.
- **Agent communication language (ACL):** a standard language for messages between agents (KQML, FIPA-ACL) using speech acts such as inform, request, propose.
- **Explainable AI (XAI):** methods that make AI decisions understandable to humans.

💡 Core concepts

Natural language processing

- **Levels of analysis:** phonology, morphology (word structure), lexical (tokenization, POS tagging), syntax (parsing with grammars), semantics (meaning), pragmatics (context and intent), discourse.
- **Ambiguity:** lexical ("bank"), syntactic ("I saw the man with a telescope"), semantic, referential (pronouns).
- **Techniques:** n-gram language models with smoothing; text classification (naive Bayes, neural); parsing with probabilistic CFGs; word embeddings (word2vec); transformers and large language models.
- **Machine translation:** rule-based (dictionaries and grammar rules), statistical MT (alignment models learned from parallel corpora, phrase-based), neural MT (encoder-decoder with attention, transformers, as in Google Translate). Evaluation by BLEU score.

Information retrieval

- Components: document collection, indexing (inverted index), query processing, ranking, results.
- Models: Boolean model, vector space model with TF-IDF weights and cosine similarity, probabilistic models (BM25), PageRank for link analysis.
- Evaluation: precision, recall, F-measure, mean average precision.

Information extraction: named entity recognition (person, organization, location, date), relation extraction ("works for"), event extraction, template filling; approaches: finite-state automata and regular expressions, probabilistic models (HMM, CRF), and neural models.

Robotics

- **Hardware:** sensors (range finders, sonar, LIDAR, cameras, GPS, IMU, force sensors), effectors (wheels, legs, grippers, arms), degrees of freedom.
- **Perception:** estimating the robot's state and the world from noisy sensors: localization (Monte Carlo localization with particle filters), mapping, SLAM, Kalman filters.
- **Planning:** configuration space (C-space), motion planning methods: cell decomposition, visibility graphs, potential fields, probabilistic roadmaps, rapidly-exploring random trees (RRT).
- **Motion and control:** PID controllers, trajectory following, handling uncertainty.
- Applications: manufacturing, warehouses, autonomous vehicles, surgical robots, drones, space and underwater exploration.

Speech recognition: pipeline: audio signal → feature extraction (MFCC) → acoustic model (HMM-GMM historically, now deep neural networks) → pronunciation model → language model → decoding (most likely word sequence, Viterbi). End-to-end deep models (CTC, transformers) are now common. Challenges: noise, accents, homophones, continuous speech, speaker variation.

Software agents

- **Architecture:** reactive, deliberative (BDI: beliefs, desires, intentions), hybrid.
- **Communication:** ACLs (KQML, FIPA-ACL) with performatives (inform, request, agree, refuse, propose); shared ontologies so messages are understood; protocols such as the Contract Net.
- **Trust:** agents need to judge whether others will behave as expected: direct experience, reputation from others, certificates, institutional rules.

Multi-agent negotiation and reputation

- **Negotiation:** agents reach agreements over the division of resources or tasks: auctions (English, Dutch, first-price sealed-bid, Vickrey/second-price sealed-bid where truthful bidding is optimal), bargaining (alternating offers), Contract Net protocol (manager announces task, contractors bid, manager awards).
- **Game theory concepts:** dominant strategy, Nash equilibrium (prisoner's dilemma), Pareto optimality, mechanism design.
- **Reputation systems:** agents rate each other after interactions (eBay feedback, peer ratings); aggregated scores guide future partner selection; must resist fake ratings and collusion.

Explainable AI (XAI)

- **Why it matters:** trust, accountability, debugging, fairness and bias detection, legal requirements (right to explanation), safety in high-stakes areas (medicine, finance, law).
- **Interpretable models:** decision trees, linear models, rule lists (transparent by design).

- **Post-hoc techniques for black-box models:** feature importance; LIME (fits a simple local model around one prediction); SHAP (Shapley values from game theory, fair contribution of each feature); partial dependence plots; counterfactual explanations ("the loan would be approved if income were ₹5,000 higher"); saliency maps, Grad-CAM for images; attention visualization for text.
- **Case studies:** NLP: highlighting words that drove a sentiment or toxicity classification, revealing bias (for example identity terms wrongly flagged); vision: Grad-CAM showing a "husky vs wolf" classifier actually used snow in the background, or a medical model focusing on hospital markers instead of the lesion.
- **Trust in AI:** explanations should be faithful (reflect the real model), understandable to the user, and actionable; combined with fairness audits and human oversight.

🧮 Solved examples

Example 1 (precision and recall). A search returns 20 documents, of which 15 are relevant; the collection has 30 relevant documents. Precision = 15/20 = **0.75**; recall = 15/30 = **0.50**; F1 = 2 × 0.75 × 0.5/1.25 = **0.60**.

Example 2 (bigram language model). Corpus: "I like AI. I like ML. I study AI." P(like | I) = count(I like)/count(I) = 2/3 ≈ **0.67**; P(study | I) = 1/3.

Example 3 (TF-IDF). Term "robot" appears 3 times in a document; the collection has 1,000 documents and "robot" appears in 10 of them. TF = 3; IDF = log10(1000/10) = 2; TF-IDF = **6**.

Example 4 (Vickrey auction). Bids: A ₹500, B ₹450, C ₹300. A wins and pays the second-highest bid, **₹450**. Bidding one's true value is a dominant strategy, so agents need not strategize.

Example 5 (XAI with SHAP, conceptually). A loan model predicts rejection with score 0.72 against an average of 0.40. SHAP values: low income +0.20, high existing debt +0.15, short credit history +0.05, good repayment record −0.08. The values add up to the difference 0.32, showing which features drove the decision.

Key points to remember

- NLP levels and ambiguity; MT: rule-based, statistical, neural; BLEU.
- IR: inverted index, TF-IDF, cosine similarity, precision and recall. IE: NER, relation extraction.
- Robotics: sensors, effectors, localization, SLAM, C-space, RRT, PID.
- Speech: MFCC, acoustic model, language model, decoding.
- Agents: BDI, ACL (KQML, FIPA), trust; negotiation (auctions, Contract Net); reputation.
- XAI: interpretable models, LIME, SHAP, counterfactuals, Grad-CAM.

⭐ Important questions

1. Explain NLP: levels of analysis, ambiguity and language models. Explain machine translation approaches.
2. Explain information retrieval and information extraction with evaluation measures.
3. Explain robotics: hardware, perception, planning and motion.
4. Explain speech recognition.

5. Explain software agents: architecture, communication and trust.
6. Explain multi-agent negotiation and reputation mechanisms.
7. What is explainable AI? Why is interpretability important? Explain techniques with case studies.

Last-hour checklist

- ☐ AI definitions and four approaches; Turing test; history table; applications.
- ☐ Agent, rationality, PEAS (taxi), environment types table, five agent types.
- ☐ Problem formulation: water jug, 8-puzzle.
- ☐ Uninformed search table; BFS and DFS orders.
- ☐ Greedy vs A* (cost 10 vs 8 example); admissible and consistent heuristics; h1 and h2 for the 8-puzzle.
- ☐ Hill climbing problems; simulated annealing probability.
- ☐ CSP: backtracking, MRV, degree, LCV, forward checking, AC-3; map colouring.
- ☐ Minimax and alpha-beta (root 3, pruned 4 and 6); expectiminimax.
- ☐ Wumpus world; propositional resolution; FOL and CNF steps; unification; forward vs backward chaining; Prolog; ontology.
- ☐ Bayes (15.4%), Bayesian network (0.000628); fuzzy operations; perceptron (AND) and XOR; backpropagation; supervised vs unsupervised.
- ☐ NLP, MT, IR (precision 0.75, recall 0.5), IE, robotics, speech, agents, negotiation (Vickrey), XAI (LIME, SHAP).

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Artificial Intelligence: A Modern Approach (4th ed.)	Stuart Russell, Peter Norvig	Pearson (2022)	WorldCat
2	Prolog Programming for Artificial Intelligence (4th ed.)	Ivan Bratko	Addison-Wesley / Pearson	WorldCat
3	The Quest for Artificial Intelligence	Nils J. Nilsson	Cambridge University Press	WorldCat
4	Artificial Intelligence: Foundations of Computational Agents	David L. Poole, Alan K. Mackworth	Cambridge University Press	WorldCat

Poole and Mackworth's book is published free online by the authors at artint.info, and Nilsson's "The Quest for Artificial Intelligence" has a free draft on his Stanford web page. Both are legal companions to the prescribed texts.

Syllabus source: AKTU B.Tech CS/CE and CSE 4th year syllabus, 2025-26 (copy hosted by an affiliated college, PDF)